

Training Transformers (Encoder-Decoder vs. Decoder-Only)

Student ID: 23K-0594

Output format: order=YMD, separator='.', month=abbr ('25 September 2026' -> '2026.SEP.25')

Fingerprint: 53ddece0fd2f

Held-out input formats: #1 weekday + full month, DMY, plain day, 4-digit year; #2 full month, DMY, plain day, 2-digit year; #6 full month, DMY, ordinal day, 2-digit year; #16 short month, DMY, plain day, 4-digit year

Device: T4 GPU. Path: full (both experiments run).

Implementation

All nine TODOs are implemented and the checks pass. Training: encoder-decoder 800 steps, decoder-only 1500 steps.

Loss curves

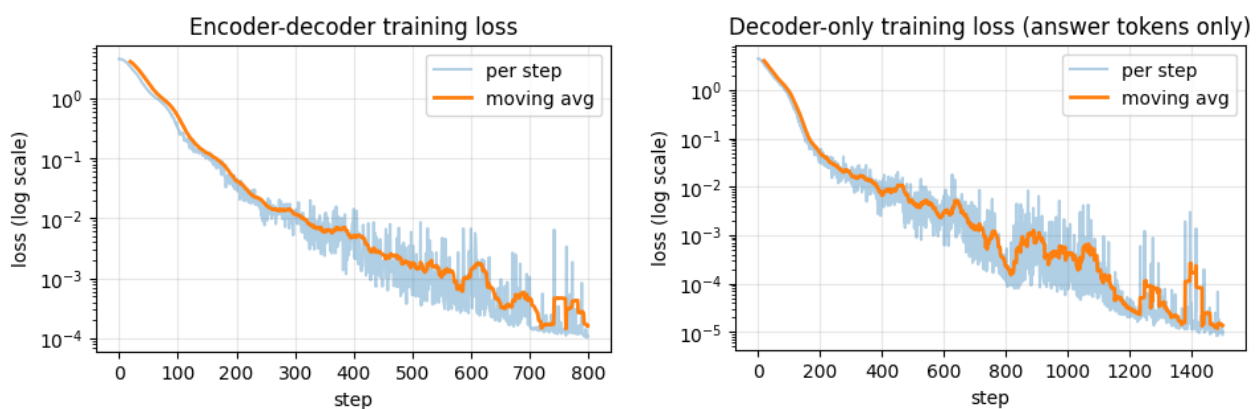


Figure 1. Training loss (log scale). Left: encoder-decoder. Right: decoder-only with answer-only loss.

Part C: Comparison table

	Encoder-decoder	Decoder-only
Parameters	953,161	1,018,569
Training steps	800	1500
Training time (s)	18	46
Time per step (ms)	23	30
Val exact match	100.0%	100.0%
Held-out exact match	100.0%	100.0%
Held-out per-field (Y/M/D)	1.000 / 1.000 / 1.000	1.000 / 1.000 / 1.000
Positions for one example	16 enc + 12 dec	29 in one sequence
Input reaches output via	cross-attention	causal self-attention

Experiment A (final train loss / val exact match)

Run	Final train loss	Val exact match
control (both masks)	0.0040	100.0%
no causal mask	0.0011	0.0% (outputs like '1911.AUG.11')
no padding mask	0.0080	98.0%

Experiment B (decoder-only)

	Answer-only loss	Every-token loss
Final train loss	0.0000	0.5622
Val exact match	100.0%	100.0%
Held-out exact match	100.0%	100.0%

Heatmaps

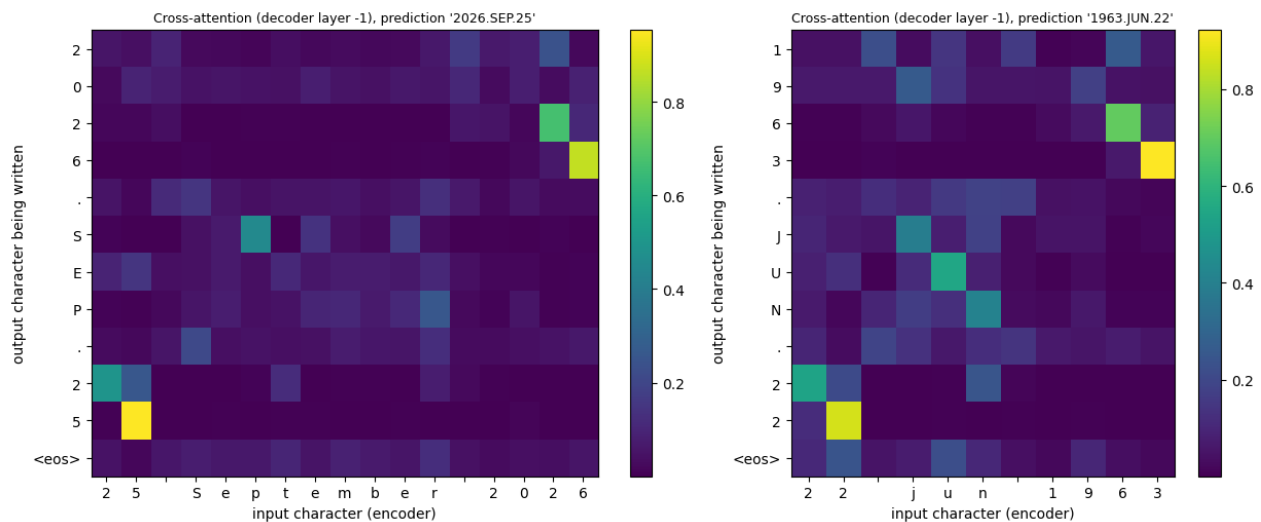


Figure 2 (left): cross-attention for '25 September 2026' -> '2026.SEP.25'. Figure 3 (right): held-out example '22 jun 1963' -> '1963.JUN.22'.

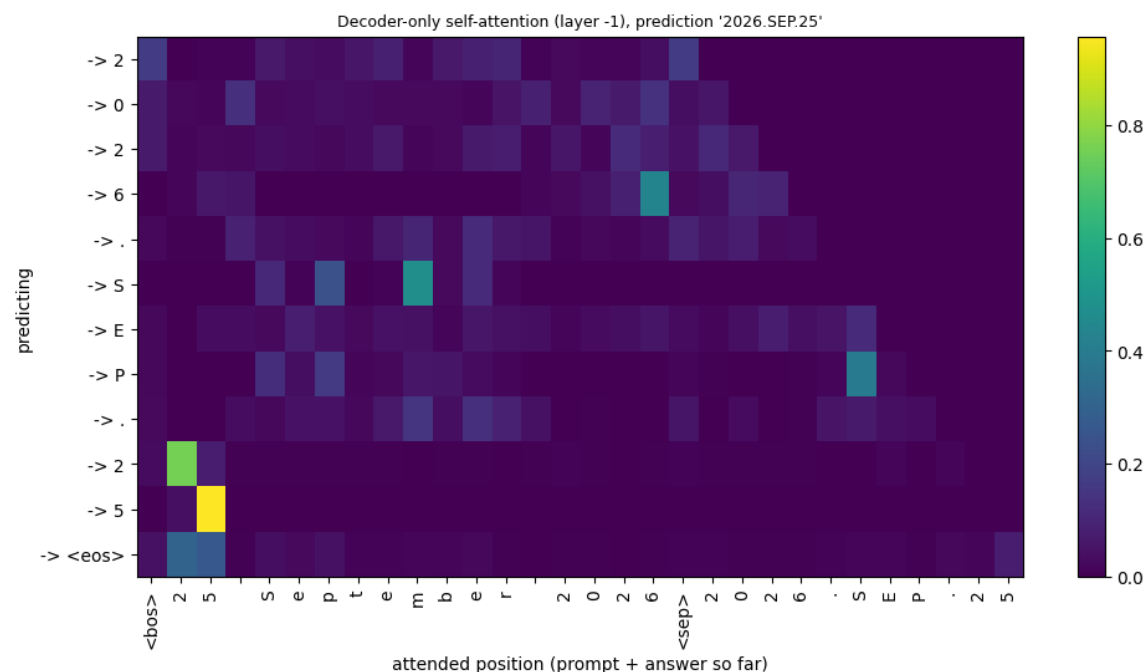


Figure 4. Decoder-only self-attention for '25 September 2026'.

Answers:

R1. Each score is a dot product of d_k numbers, so it gets larger as d_k grows. Large scores push the softmax toward one-hot, where its gradient is close to zero and learning stalls. Dividing by $\sqrt{d_k}$ keeps the scores in a normal range so the softmax stays smooth.

R2. Figure 2 (output order Y, M, D). Year (2, 0, 2, 6): the last two digits look sharply at the input '2' (about 0.6) and '6' (about 0.85); the first two digits are spread out, because every year starts with 19 or 20. Month (S, E, P): the weight is spread over the letters of 'September', highest on 'p' for S, with no single clear cell.

That fits a lookup from the month name to 3 letters, not a copy. In the held-out example (Figure 3) J, U and N look at j, u and n in turn. Day (2, 5): '2' attends to the input '2' (about 0.55) and '5' to the input '5' (about 0.95), which is a direct copy. Separators and <eos> are faint. Because the input is DMY and the output is YMD, the bright cells run from the end of the input to the start instead of along a diagonal.

R3. Encoder-decoder: validation 100.0%, held-out 100.0%, per-field Y 1.000, M 1.000, D 1.000. Decoder-only: the same numbers. No mistakes were printed, so I cannot say from the errors which field was hardest. I expected the year, because of the 2-digit years, but held-out formats #2 and #6 have 2-digit years and were still 100%. The only wrong outputs I saw came from the broken models in Experiment A (for example '1911.AUG.11').

R4. Without the causal mask, decoder position i can look at later positions, and the label for position i is exactly the input token at position $i+1$. The model just copies it, which gives a lower training loss than the control (0.0011 vs 0.0040) without learning the task. At generation time the later tokens do not exist (only <bos> and the model's own earlier outputs), so the shortcut is gone and exact match is 0.0%, with outputs like '1911.AUG.11'.

R5. Without the padding mask, an input like '1 Aug 99' (8 characters) is followed by 32 <pad> tokens that all receive attention weight. It still works because <pad> always has the same embedding, so the model can learn to give it keys with low scores and a value that does no harm. That is extra work, which explains the slower learning: final loss 0.0080 vs 0.0040 and accuracy 98.0% vs 100.0% at 400 steps. The shortest inputs suffer most, since most of their window is padding.

R6. Causal self-attention over the prompt positions does the job of cross-attention: the answer positions look back at the prompt in the same sequence. In Figure 4 the day digits '2' and '5' attend to the prompt '2' and '5' (about 0.75 and 0.95), and the last year digit '6' attends to the '6' of 2026. For the month, S attends mostly to the 'm' of 'September', and P attends to the 'S' the model has just written, so it also uses its own earlier output. <eos> attends to the day digits at the start of the prompt. The map is an average over heads in the last layer, so it is blurrier than the cross-attention map.

R7. The every-token loss (0.5622 vs 0.0000) includes predicting the messy input characters. What can never be predicted is the input itself: the random day, month and year, the letter case, the weekday and which of the formats was used. The answer is fully determined by the input, so only the answer loss goes to zero. Predicting the input forces the model to learn how dates are written, which could help on unseen formats, but in my run both models were already at 100% on validation and held-out, so I saw no gain. Every-token loss corresponds to pre-training and answer-only loss to SFT. Instruction tuning masks the prompt so the model learns to produce the response and is not trained to imitate user prompts.

R8. For one example the encoder-decoder processes $16 + 12 = 28$ positions in two small stacks (2 encoder and 2 decoder layers), while the decoder-only model processes 29 positions in one sequence through 5 layers. Attention cost grows with the square of the length and all 5 layers work over the whole sequence, so a decoder-only step costs more (in training the sequences are padded: $40 + 16$ for the encoder-decoder, 56 for the decoder-only model). Measured time per step is 23 ms vs 30 ms (1.3x), and 18 s vs 46 s in total because the decoder-only model also needed more steps (1500 vs 800). The measured gap is smaller than the counts suggest, probably because this model is small and the fixed cost per GPU operation dominates.

R9. Table above. For my date task I would choose the encoder-decoder: both models reach 100% on validation and held-out data, but it trains in 18 s instead of 46 s, needs 800 steps instead of 1500, and has fewer parameters. LLMs are decoder-only for two reasons. First, one sequence format covers everything, so there is no decision about what is input and what is output, and every token is a training signal. Experiment B fits this: the same decoder-only model trained on every token, input included, still reached 100% on validation and held-out. Second, conversations and agent runs keep growing, and a decoder-only model can just append to one sequence and reuse its cached keys and values, while an encoder-decoder would have to re-encode the input each time it grew.

R10. Held-out exact match is 100.0% for both models, so the difference is 0 and neither generalised better. The held-out formats are new combinations of pieces that both models saw in training (full months, DMY order, 2-digit years), and both read the input through attention over all of its characters, so a new format only changes which characters get attended to. The held-out heatmap shows this: the encoder-decoder still matches J, U, N with j, u, n. To separate the two models I would need a harder test, such as years outside the training range.

R11. The decoder-only setup is closer. The user's request and the tool descriptions form the prompt, then a separator, and then the JSON tool call is generated token by token, which is the same layout as '<bos> src <sep> answer <eos>'. Copying 'Karachi' from the prompt into the call is like my model copying the day digits from the prompt. Models are trained for this with answer-only loss, like my masked labels (no loss on prompt positions, loss from the <sep> position onward). When the tool's result is appended, the sequence is just longer and the model continues from it using cached keys and values, with no separate encoder to run again.

Reflection

The prediction I got most wrong was that the year would be the hardest field. The model learned the 2-digit-year rule from the training data and got every held-out year right. Cross-attention and causal self-attention differ in that cross-attention takes its keys and values from a different sequence (the encoder output), while causal self-attention takes them from earlier positions of the same sequence. One thing I now understand about the models behind my agents is that a tool call is just answer tokens after a prompt, produced by the same causal attention that reads the prompt and any tool results added later.